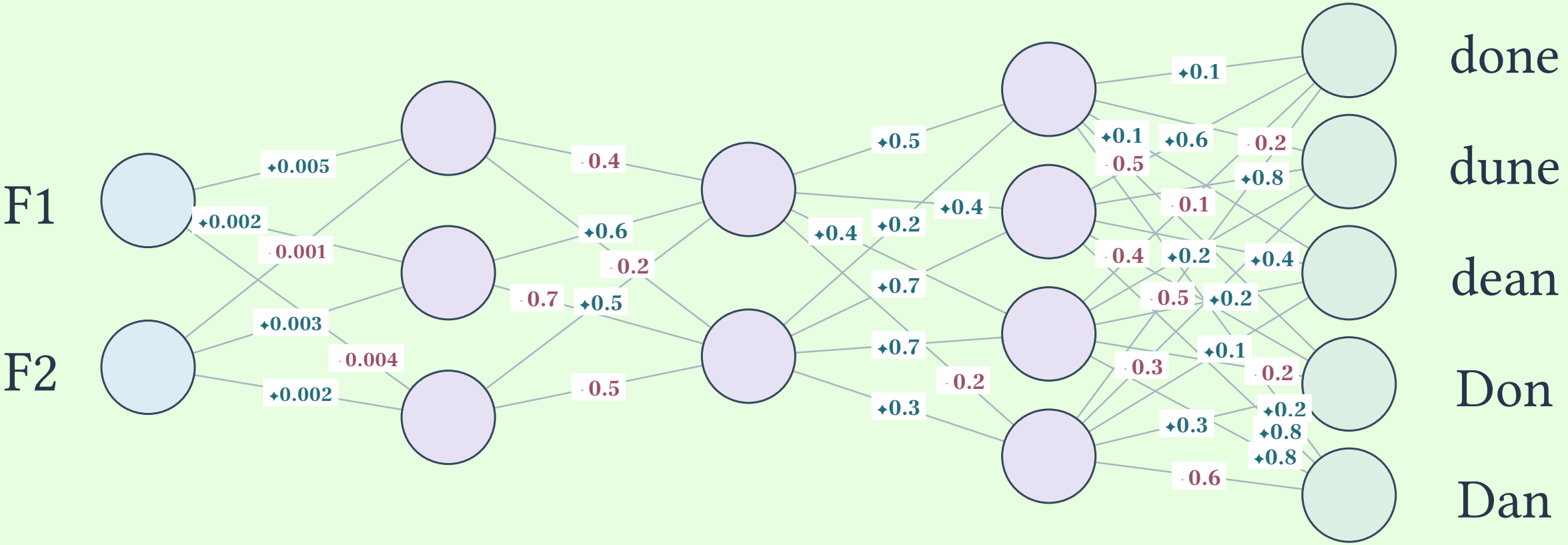
Ling 285
Human Language and Technology
Lecture 16

Neural Networks

Khalil Iskarous

Department of Linguistics

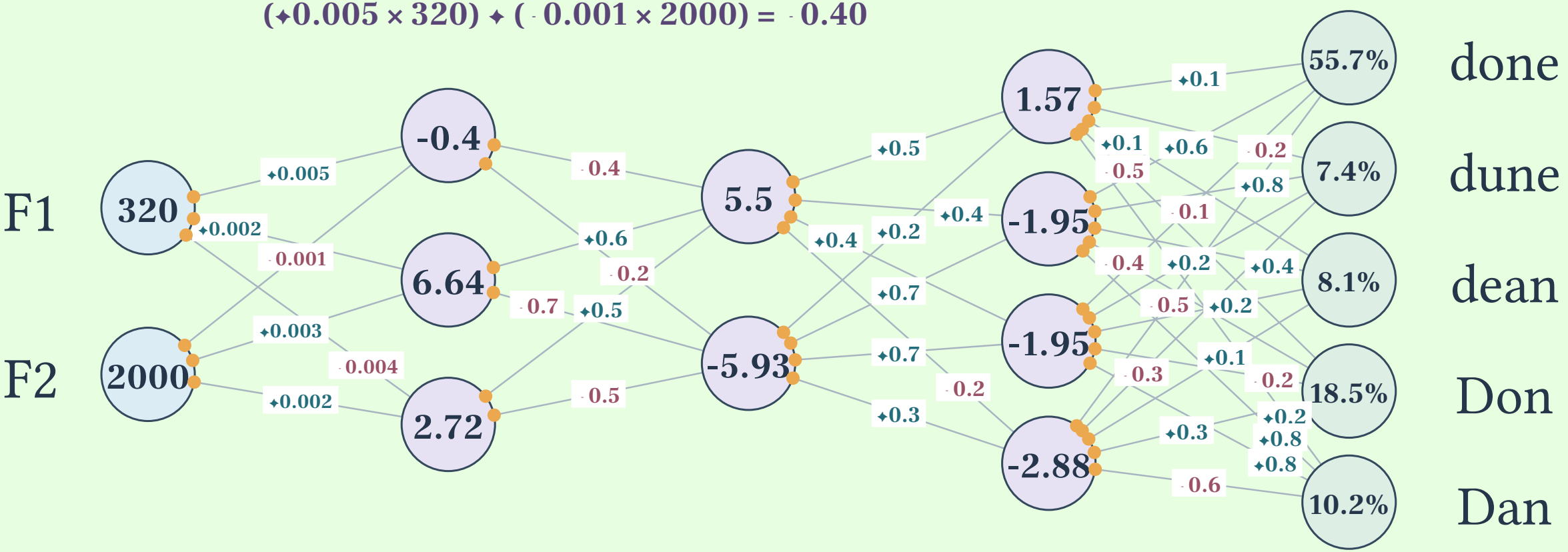
University of Southern California


F1
F2
+0.005
+0.002
-0.001
+0.003
-0.004
+0.002
-0.4
+0.6
-0.2
-0.7
+0.5
-0.5
+0.5
+0.4
+0.2
+0.4
+0.7
+0.7
-0.2
+0.3
+0.1
+0.1
+0.6
-0.2
-0.5
+0.8
-0.1
-0.4
+0.2
+0.4
-0.5
+0.2
+0.1
-0.3
-0.2
+0.2
+0.3
+0.8
+0.8
-0.6
done
dune
dean
Don
Dan

Forward propagation

Top neuron: $w_1F1 + w_2F2$

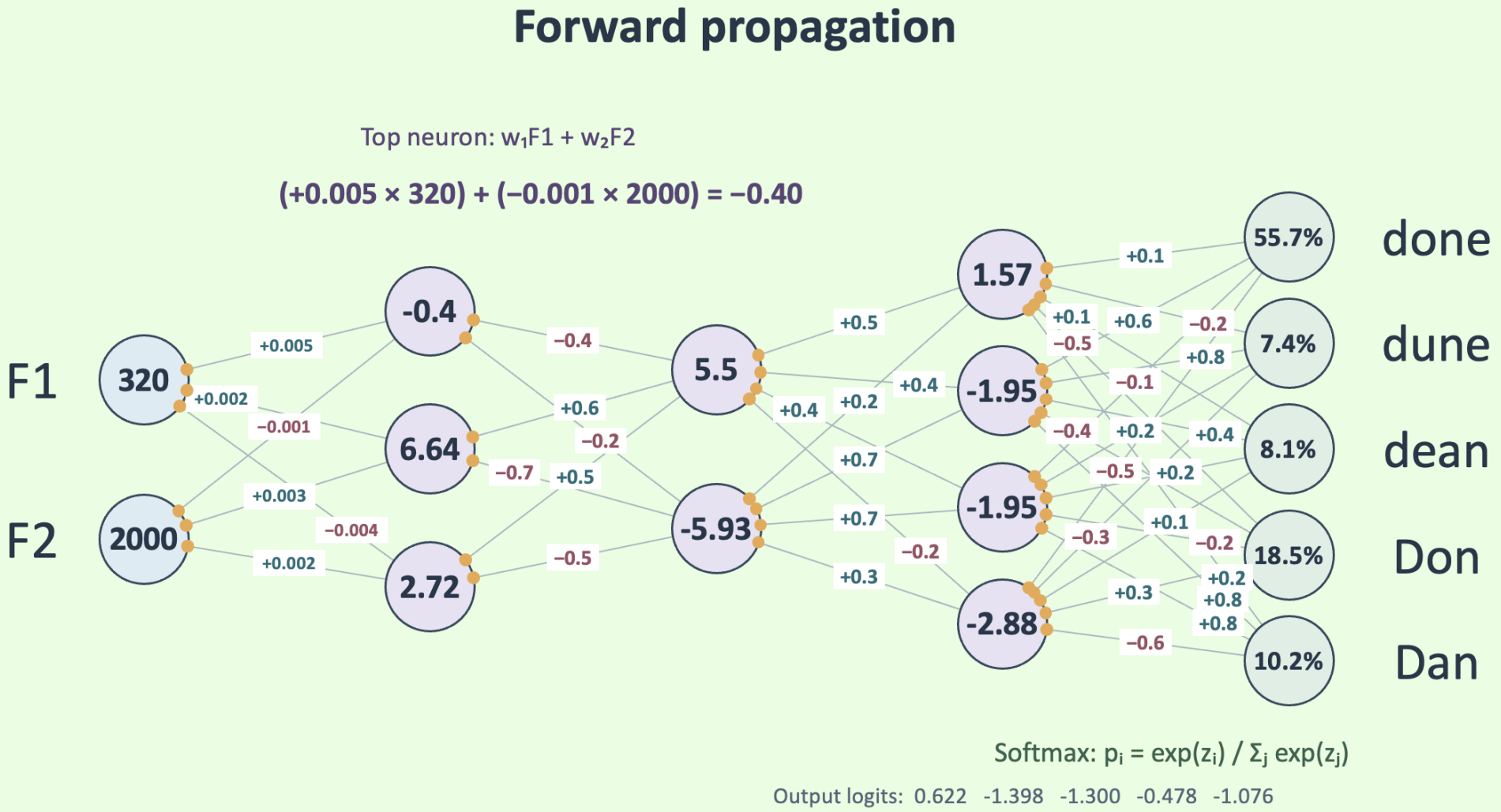
$$(+0.005 \times 320) + (-0.001 \times 2000) = -0.40$$



Softmax: $p_i = \exp(z_i) / \Sigma_j \exp(z_j)$

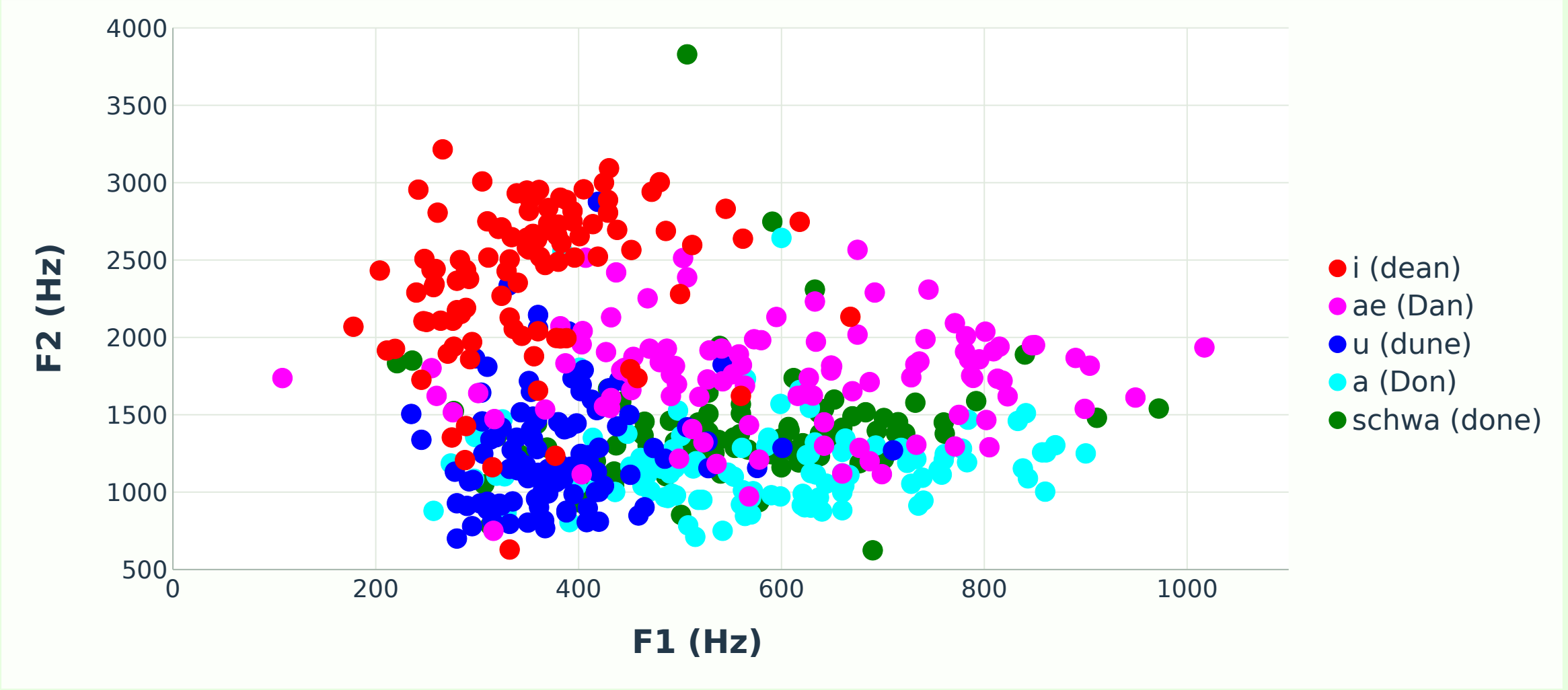
Output logits: 0.622 −1.398 −1.300 −0.478 −1.076

Pop Quiz!


Forward propagation
Top neuron: $w_1F1 + w_2F2$
(+0.005 × 320) + (−0.001 × 2000) = −0.40
F1
F2
320
2000
-0.4
6.64
2.72
5.5
-5.93
1.57
-1.95
-1.95
-2.88
55.7%
7.4%
8.1%
18.5%
10.2%
done
dune
dean
Don
Dan
Softmax: $p_i = \exp(z_i) / \Sigma_j \exp(z_j)$
Output logits: 0.622 -1.398 -1.300 -0.478 -1.076

Vowel formants

525 observations, with 105 tokens of each vowel



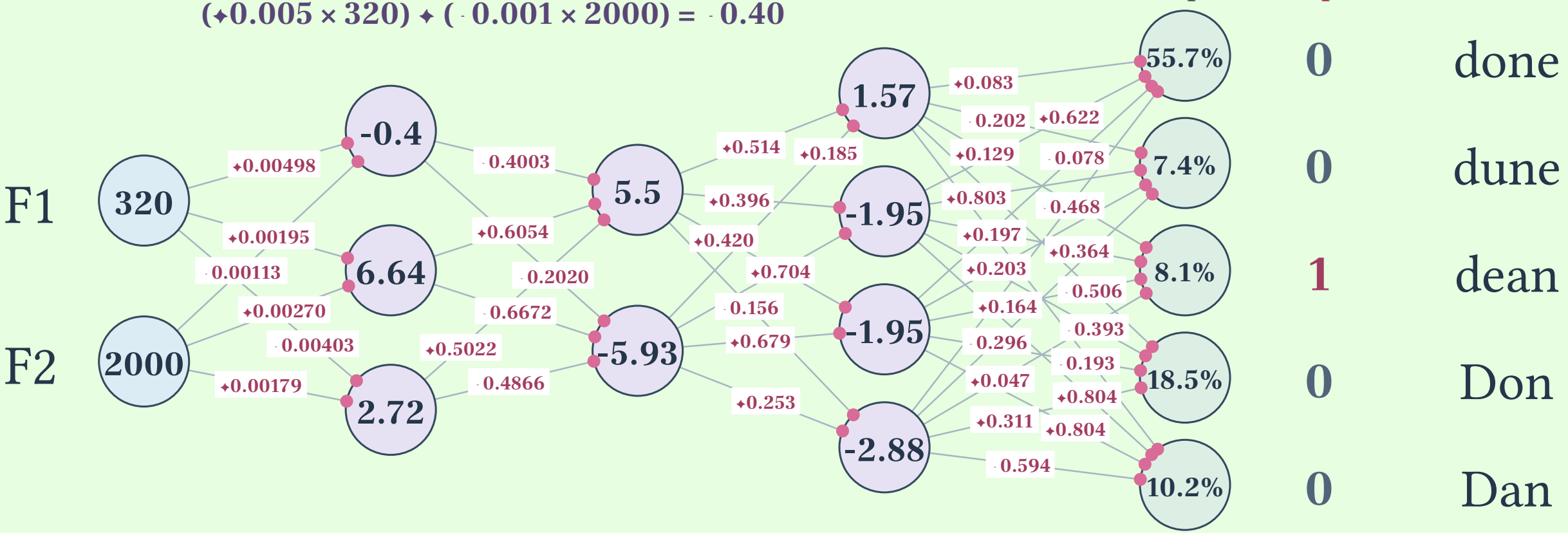
All rows from formant_information.csv. F1 is horizontal and F2 is vertical, in Hz.

Info on Exam 2 and Exam 3

Backpropagation

Top neuron: $w_1F1 + w_2F2$

$$(+0.005 \times 320) + (-0.001 \times 2000) = -0.40$$



Weight update: $W \leftarrow W - \eta \, \partial L/\partial W$

Softmax: $p_i = \exp(z_i) / \Sigma_j \exp(z_j)$

Cross-entropy for dean. Forward activations stay fixed during this backward pass.

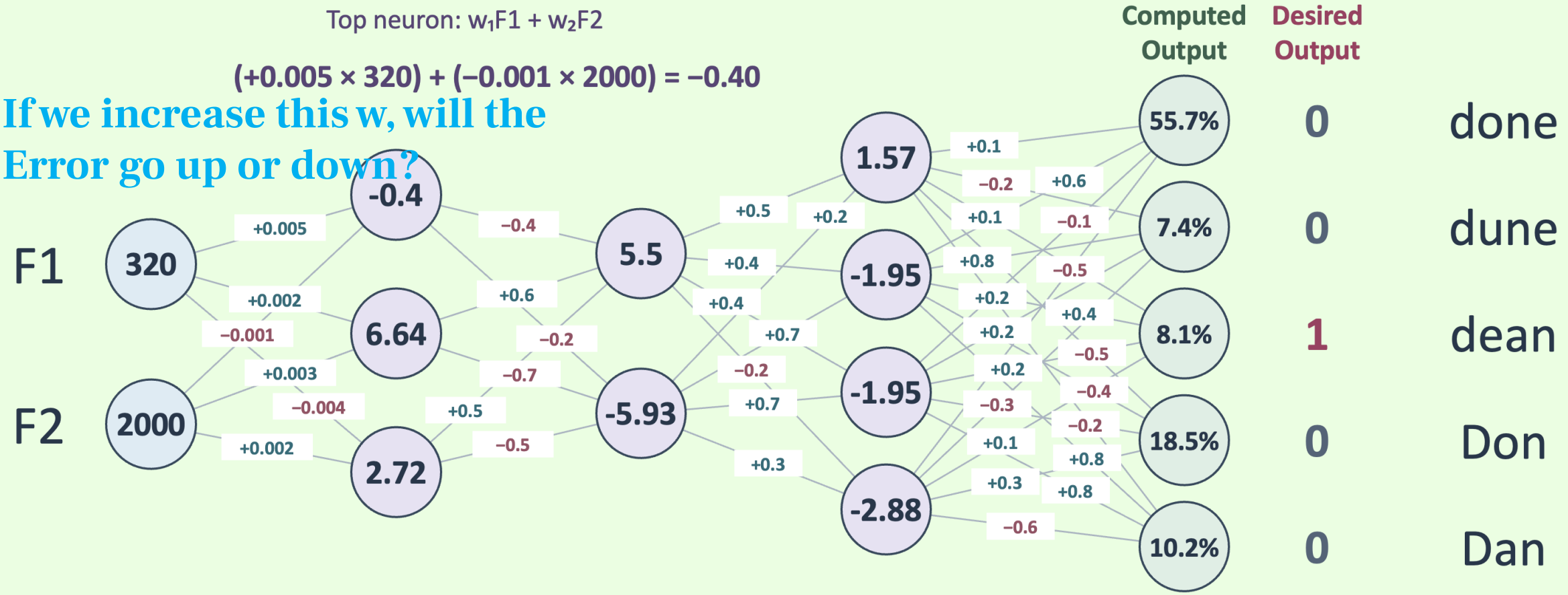
Backpropagation

If we increase this w, will the Error go up or down?

Top neuron: $w_1F1 + w_2F2$

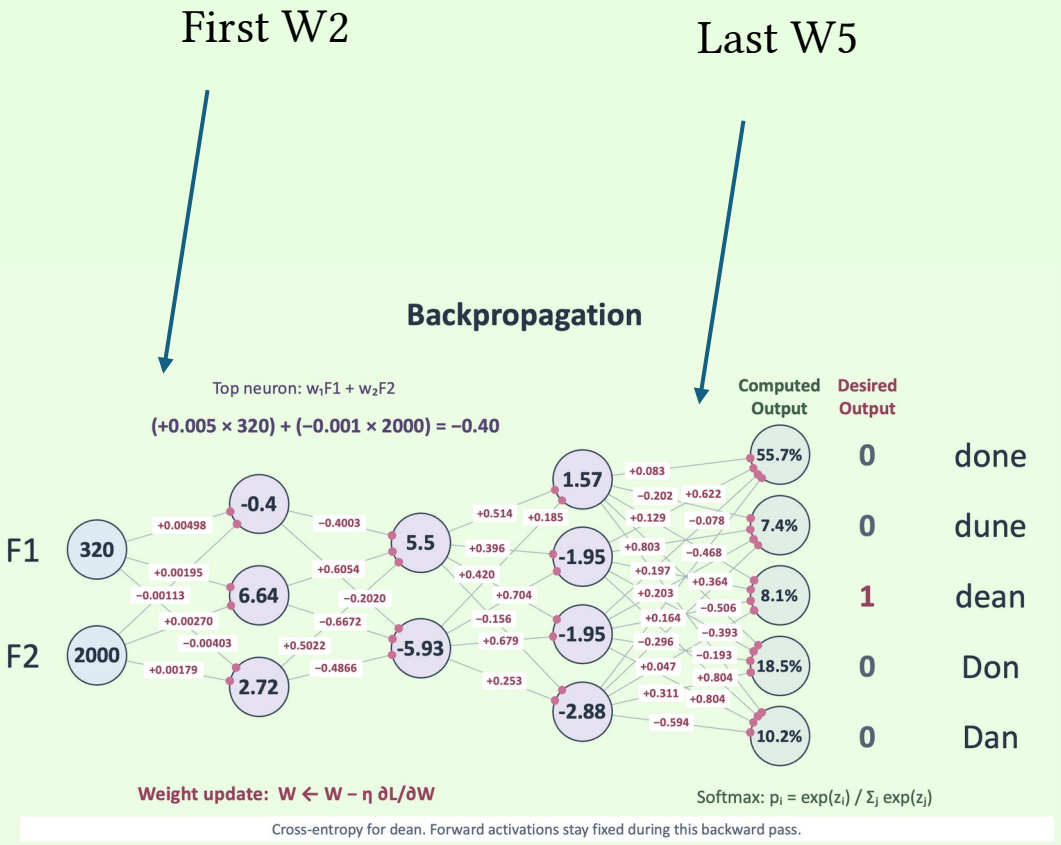
$$(+0.005 \times 320) + (-0.001 \times 2000) = -0.40$$

If we increase this w, will the Error go up or down?



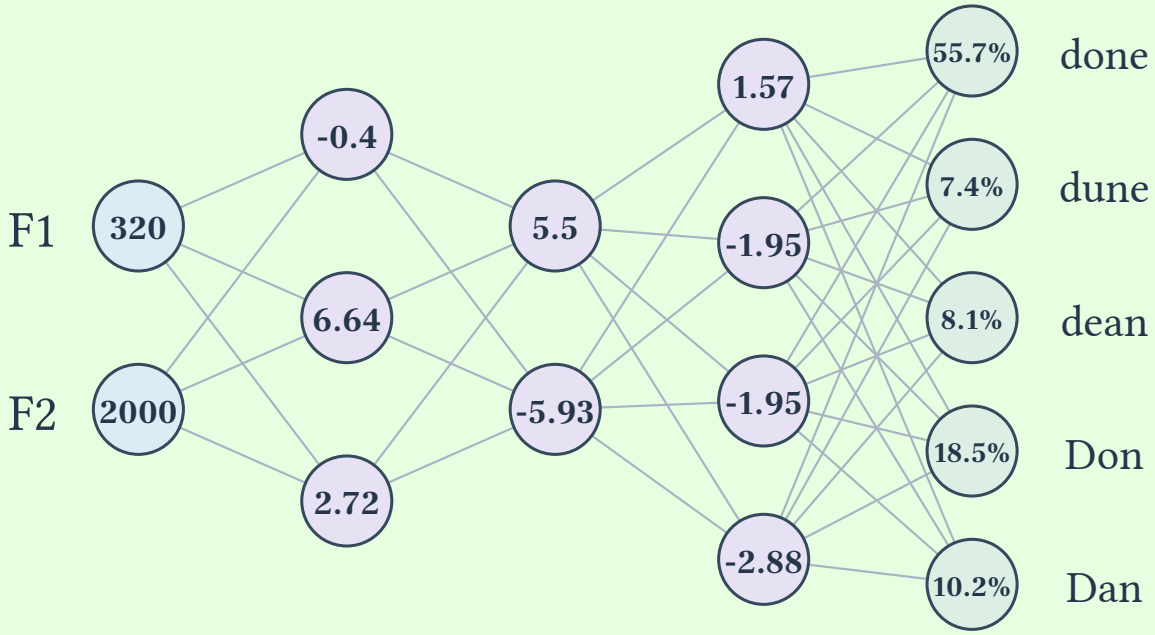
Why do we *back*propagate?

- Let's think about the problem of w-modification, and let's compare the last set of w's and the first
- Last W5: The output probabilities are directly determined by the Last w's!!!! The error is very directly determined by the last W's
- We can use the error between computed and desired directly to determine better w's.
- First W2 affects its activations, which affects another 2 layers of activations along with other W's. It's really hard to figure out the culpability of W2 for the mistakes made all the way at the end!!!!! Moreover, the effect of W2 on errors INCLUDES the effect of W5 on them, since W5 is downstream of W2!!!!
- So which problem is easier: Improving W2 or W5?
- W5 is easy! What algorithm do we use to solve small problems then bigger ones that depend on them?
- Dynamic programming?
- *Back*propagation is Bellman-Viterbi Dynamic Programming!!!
- Backpropagation = SGD + Dynamic programming ☺



Training matrices

Before backpropagation



2 → 3 → 2 → 4 → 5

Formant_Inputs

110 × 2

F1	F2
500	1500
350	1000
320	**2000**
700	1100
800	1700
⋮	⋮

TrueVowel_Outputs

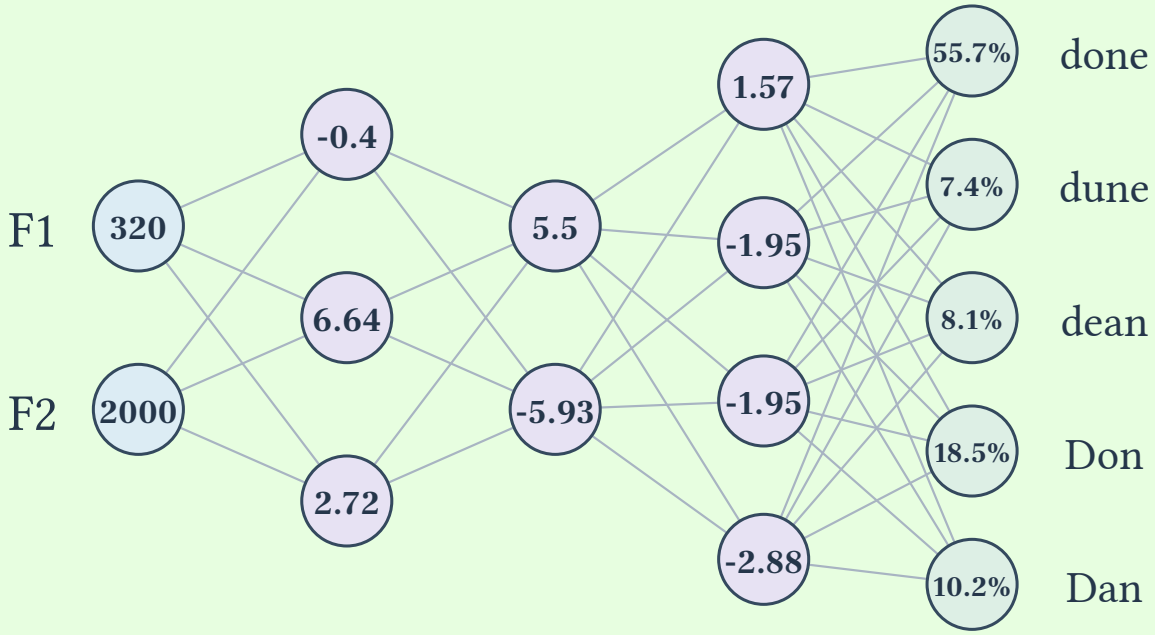
110 × 5

done	dune	dean	Don	Dan
1	0	0	0	0
0	1	0	0	0
0	**0**	**1**	**0**	**0**
0	0	0	1	0
0	0	0	0	1
⋮	⋮	⋮	⋮	⋮

Illustrative rows. Each row pairs one recording with its vowel.

The same network in PyTorch

Before backpropagation



2 → 3 → 2 → 4 → 5

```
import torch
from torch import nn

# Sizes: X = 110x2, Y = 110x5
X = torch.load("Formant_Inputs.pt").float()
Y = torch.load("TrueVowel_Outputs.pt").float()

net = nn.Sequential(
    nn.Linear(2, 3),
    nn.Linear(3, 2),
    nn.Linear(2, 4),
    nn.Linear(4, 5)
)

sgd = torch.optim.SGD(net.parameters(), lr=0.1)
sgd.zero_grad()
loss = nn.CrossEntropyLoss()(net(X), Y)
loss.backward()
sgd.step()
```

One gradient step using all 110 rows

The sigmoid activation function

Lord Edgar Adrian

Nobel Prize in Physiology or Medicine, 1932

Neurons are nearly digital! On or off, roughly.
If the inner product is large positive,
sigmoid function is near 1, and if inner product
Is large negative, sigmois is basically 0.
Animal perception reflects this: some things that seem alike, are perceived different, and some things that are Very different are perceived same!

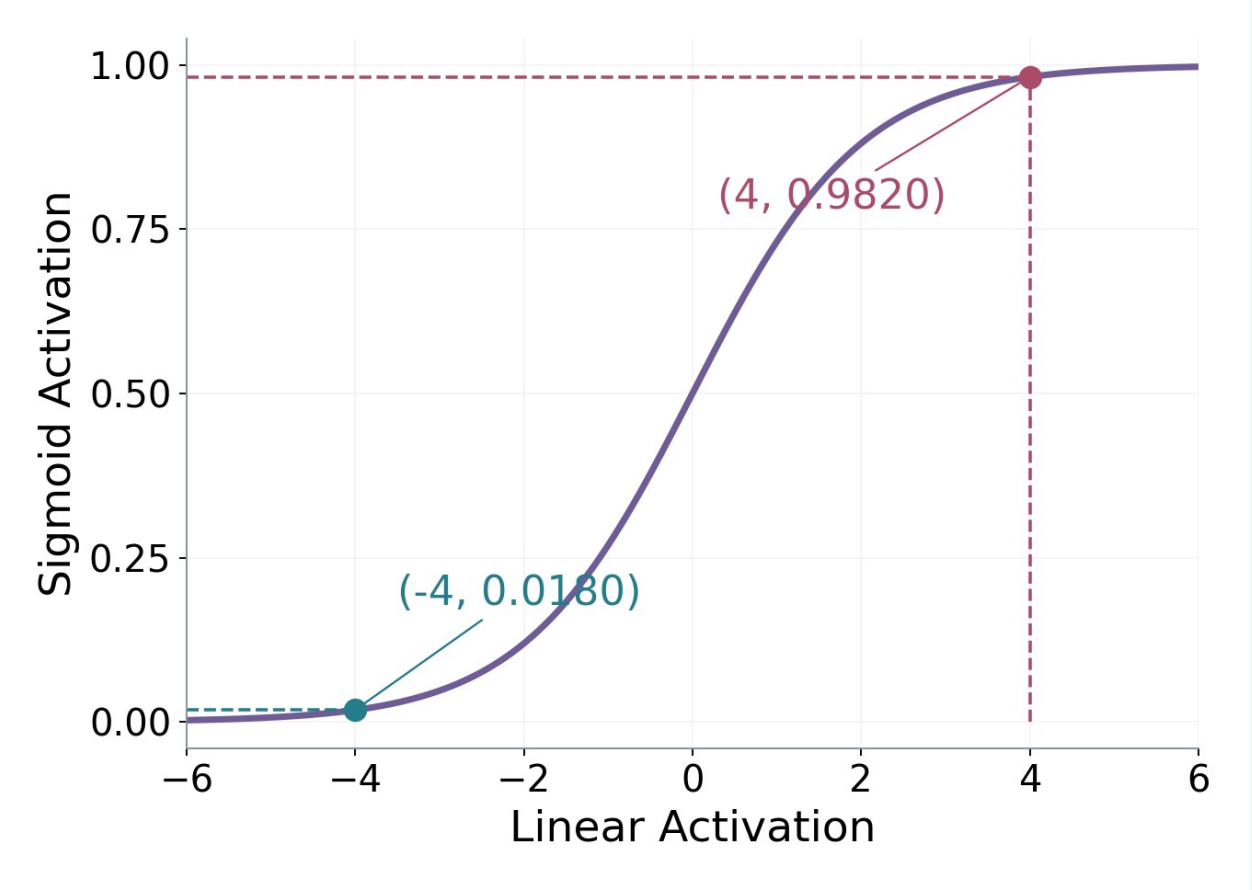
If the inner product is −4:

$$\sigma(-4) = 0.0180$$

If the inner product is 4:

$$\sigma(4) = 0.9820$$

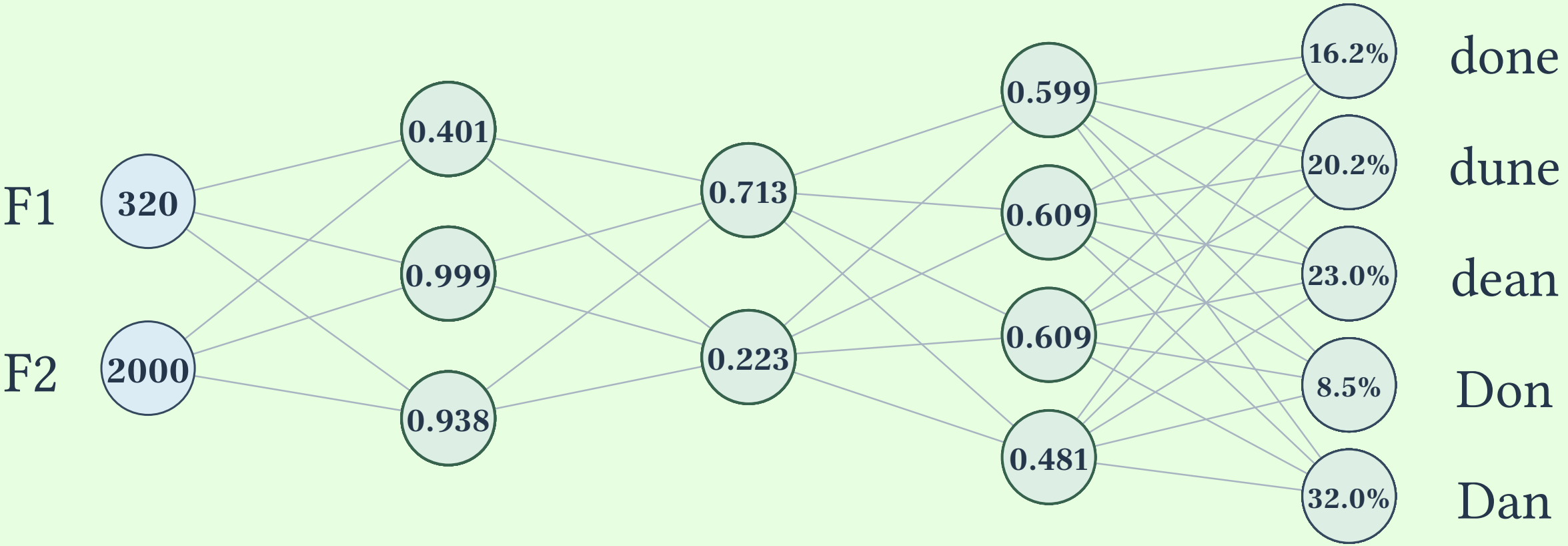
$$\sigma(z) = 1 / (1 + \exp(-z))$$



https://www.vcalc.com/wiki/vcalc/sigmoid-function

Applying sigmoid, one neuron at a time

Hidden layer 3, neuron 4: σ(-0.076) = 0.481

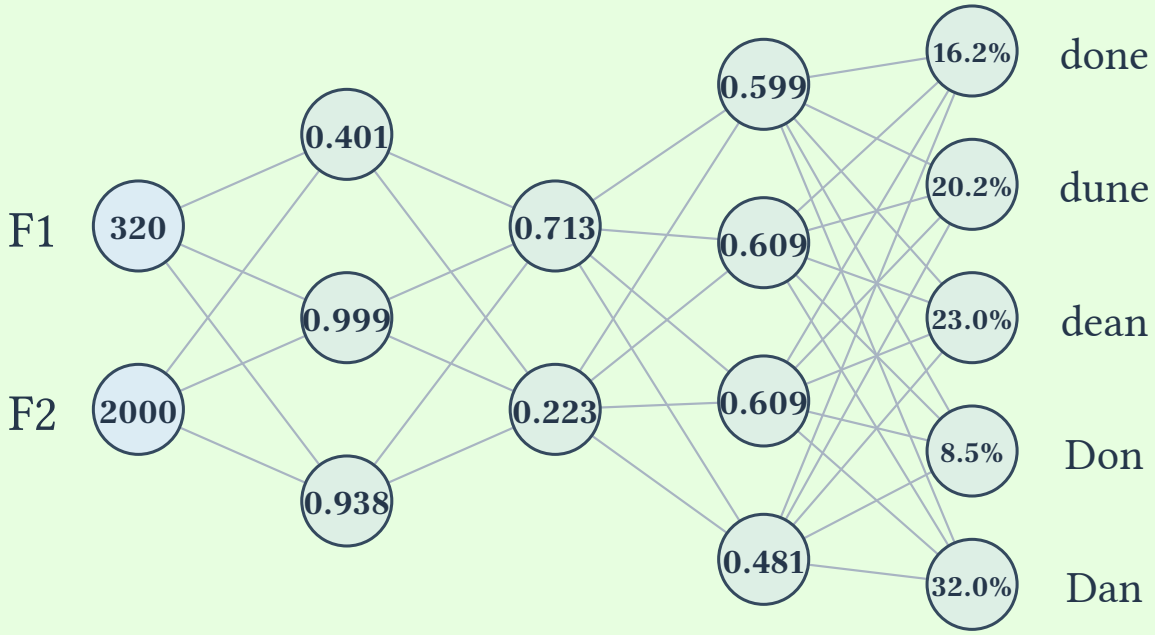


Purple: linear activation Green: sigmoid activation

Inputs stay as formants. Downstream values recalculate; outputs use softmax.

The same PyTorch program, with sigmoid

Sigmoid hidden activations



2 → 3 → 2 → 4 → 5

```
import torch
from torch import nn

# Sizes: X = 110x2, Y = 110x5
X = torch.load("Formant_Inputs.pt").float()
Y = torch.load("TrueVowel_Outputs.pt").float()

net = nn.Sequential(
    nn.Linear(2, 3), nn.Sigmoid(),
    nn.Linear(3, 2), nn.Sigmoid(),
    nn.Linear(2, 4), nn.Sigmoid(),
    nn.Linear(4, 5)
)

sgd = torch.optim.SGD(net.parameters(), lr=0.1)
sgd.zero_grad()
loss = nn.CrossEntropyLoss()(net(X), Y)
loss.backward()
sgd.step()
```

Sigmoid after each hidden layer

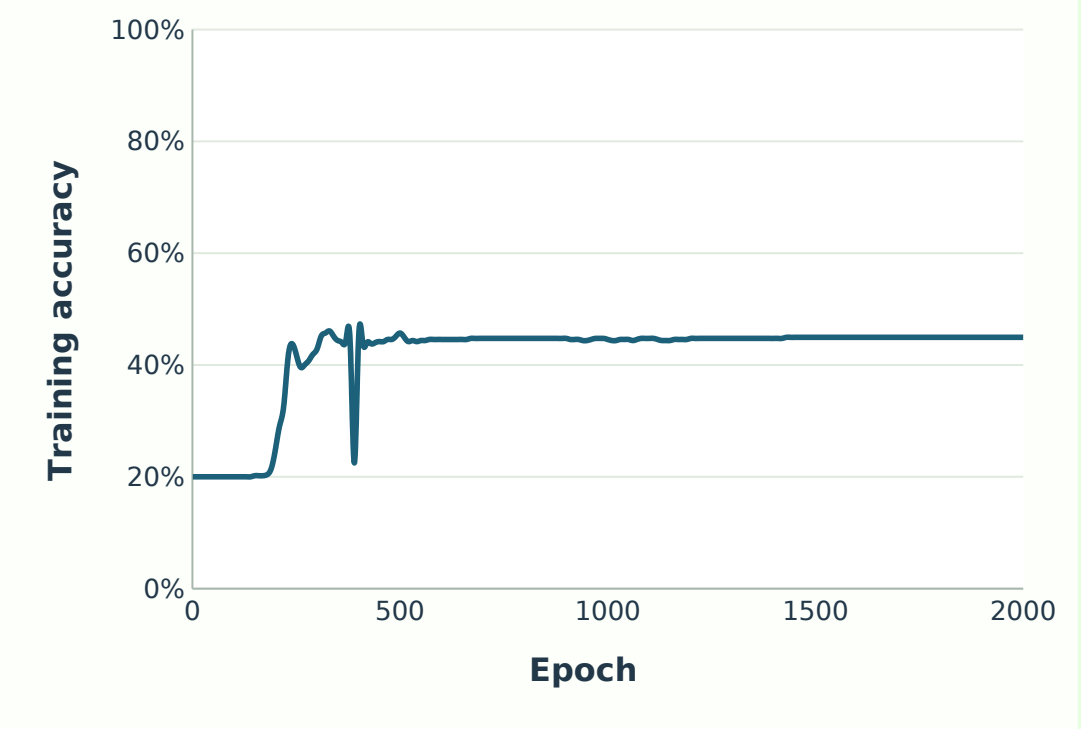
One initialization

Actual NumPy reference run, seed 0

Requested PyTorch training code

```
import torch  # Tensors
from torch import nn  # Layers and loss
import matplotlib.pyplot as plt  # Plotting
torch.manual_seed(0)  # Initialization
X = Formant_Inputs / 1000  # Hz to kHz
y = TrueVowel_Outputs.argmax(1)  # Correct classes
model = nn.Sequential(  # The supplied widths
    nn.Linear(2, 3, bias=False),  # No bias
    nn.Linear(3, 2, bias=False),  # No bias
    nn.Linear(2, 4, bias=False),  # No bias
    nn.Linear(4, 5, bias=False),  # Five scores
)  # No activations
loss_fn = nn.CrossEntropyLoss()  # One-hot cross-entropy
opt = torch.optim.SGD(model.parameters(), lr=0.1)  # Plain SGD
accuracy = []  # Percent correct
for epoch in range(2000):  # Full-batch updates
    opt.zero_grad()  # Clear old gradients
    loss = loss_fn(model(X), TrueVowel_Outputs)  # Forward + loss
    loss.backward()  # Backpropagate
    opt.step()  # Update weights
    with torch.no_grad():  # Accuracy only
        pred = model(X).argmax(1)  # Largest score wins
        correct = (pred == y).float().mean().item()  # Fraction correct
        accuracy.append(100 * correct)  # Save percent correct
plt.plot(range(1, 2001), accuracy)  # Accuracy curve
plt.ylim(0, 100)  # Full percent scale
plt.xlabel('Epoch')  # Epoch axis
plt.ylabel('Accuracy (%)')  # Accuracy axis
plt.show()  # Display the curve
```

Final training accuracy: 44.95%



2,000 full-batch updates at learning rate 0.1
Accuracy uses the same 525 training examples.

PyTorch is unavailable here. This curve comes from the equivalent NumPy implementation.

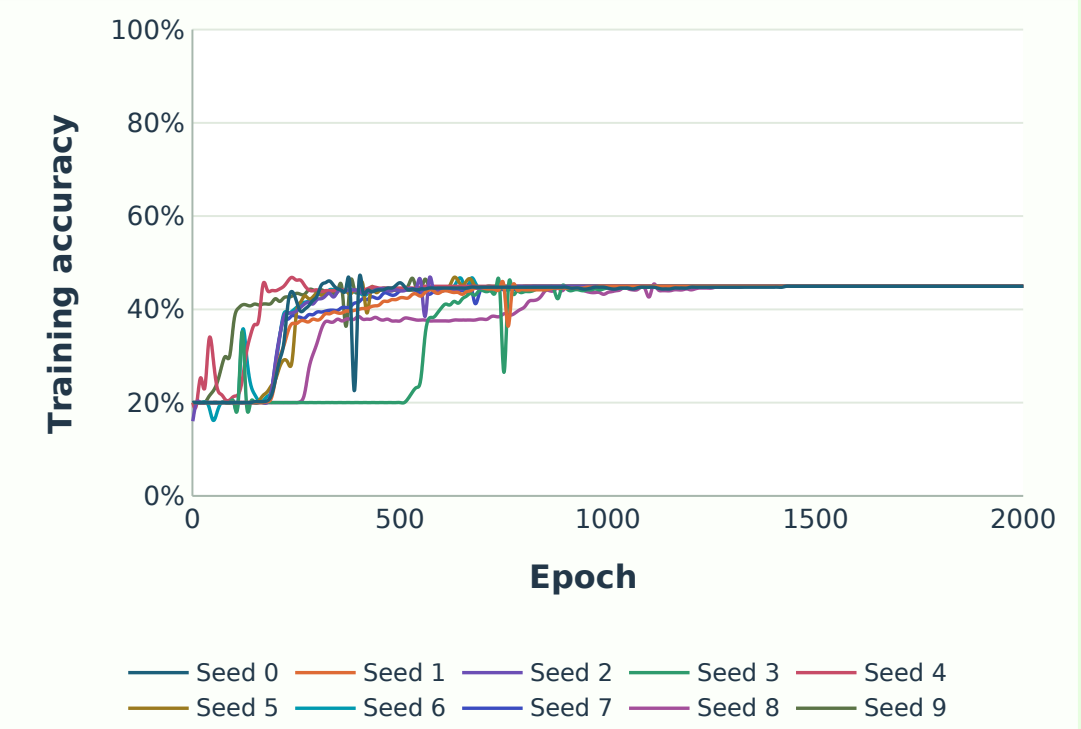
Ten initializations

Actual NumPy reference runs, seeds 0 through 9

Requested PyTorch training code

```
import torch  # Tensors
from torch import nn  # Layers and loss
import matplotlib.pyplot as plt  # Plotting
torch.manual_seed(0)  # Initialization
X = Formant_Inputs / 1000  # Hz to kHz
y = TrueVowel_Outputs.argmax(1)  # Correct classes
model = nn.Sequential(  # The supplied widths
    nn.Linear(2, 3, bias=False),  # No bias
    nn.Linear(3, 2, bias=False),  # No bias
    nn.Linear(2, 4, bias=False),  # No bias
    nn.Linear(4, 5, bias=False),  # Five scores
)  # No activations
loss_fn = nn.CrossEntropyLoss()  # One-hot cross-entropy
opt = torch.optim.SGD(model.parameters(), lr=0.1)  # Plain SGD
accuracy = []  # Percent correct
for epoch in range(2000):  # Full-batch updates
    opt.zero_grad()  # Clear old gradients
    loss = loss_fn(model(X), TrueVowel_Outputs)  # Forward + loss
    loss.backward()  # Backpropagate
    opt.step()  # Update weights
    with torch.no_grad():  # Accuracy only
        pred = model(X).argmax(1)  # Largest score wins
        correct = (pred == y).float().mean().item()  # Fraction correct
        accuracy.append(100 * correct)  # Save percent correct
plt.plot(range(1, 2001), accuracy)  # Accuracy curve
plt.ylim(0, 100)  # Full percent scale
plt.xlabel('Epoch')  # Epoch axis
plt.ylabel('Accuracy (%)')  # Accuracy axis
plt.show()  # Display the curve
```

All ten finish at 44.95%



train_ten_runs.py repeats this training block with a fresh model and SGD for each seed, 0–9.

Curves show executed NumPy reference runs. Equal seed numbers do not imply equal PyTorch weights.